

Session 3: JavaScript for QA Automation

Chapter 1: JS Programming Basics. Variables, Data Types, Control Structures, and Functions	2
Learning Objectives	2
1.1 The Great Transition: Why do we learn JavaScript?	3
1.2 The Environment: Where do we write and run JS code?	3
1.3 Variables: Memory Boxes (let vs const)	4
1.4 Data Types: What do we put in the boxes?	6
1.5 Control Structures: The Script's Intelligence	6
1.6 Functions: Reusable Recipes	7
1.7 Did you know...?	7
1.8 Applied Story: "The JavaScript Kitchen"	8
1.9 Practical Exercises	8
1.10 Exercise Solutions	9
1.11 🤖 Engineer Mindset in the AI Era: Your Place in the Market	10
Chapter 2: What do we put in the boxes? Fundamental Data Types	11
Learning Objectives	11
2.1 Primitive Types: The Basic Ingredients	11
2.2 The Mystery of "Nothingness": undefined vs null	13
2.3 Complex Structures: Arrays and Objects	13
2.4 Did you know...?	14
2.5 Practical Exercises	14
2.6 Exercise Solutions	15
2.7 🤖 Engineer Mindset in the AI Era: Your Place in the Market	15
Chapter 3: Application Logic - Control Structures (if/else, for)	17
Learning Objectives	17
3.1 Making Decisions: The if / else Statement	17
3.2 Multiple Branches: else if	19
3.3 Repetition: The for Loop	19

3.4 Iterating through Lists (Combining Loops with Arrays)	20
3.5 Did you know...?	20
3.6 Practical Exercises	20
3.7 Exercise Solutions	21
3.8 🤖 Engineer Mindset in the AI Era: Your Place in the Market	21
Chapter 4: Code Reusability - Classic Functions and Arrow Functions	23
Learning Objectives	23
4.1 What is a function and why do we need it?	23
4.2 The Classic Function	24
4.3 The Arrow Function - The Modern Standard	25
4.4 Did you know...?	25
4.5 Practical Exercises	26
4.6 Exercise Solutions	26
4.7 🤖 Engineer Mindset in the AI Era: Your Place in the Market	27
Chapter 5: Session 3 Assignment & Extra Practice (Advanced Level)	28
5.1 Homework Assignment (Session 3 Project)	28
5.2 Bonus Challenge (Optional): The API "Falsy" Trap	31
5.3 🔑 Solution Keys (Answers & Explanations)	31

Chapter 1: JS Programming Basics.

Variables, Data Types, Control Structures, and Functions

Learning Objectives

🧠 By the end of this chapter, you will be able to:

- Understand the role of JavaScript (JS) in automated testing (Playwright) and know the

environments where you can run it (IDE vs. Browser).

- Declare variables using `let` and `const` and understand the vital difference between them.
- Identify and use primitive data types (String, Number, Boolean).
- Control the execution flow using decision-making statements (if/else) and loops (for).
- Write reusable code blocks using classic functions and Arrow Functions.

1.1 The Great Transition: Why do we learn JavaScript?

If HTML is the "skeleton" of the house, and CSS is the "design", **JavaScript (JS)** represents the "electrical system and plumbing". JS transforms a static web page into an interactive application.

For you, as a future QA Automation Engineer, JavaScript is *the language you will use to speak to your robot*. Playwright and Cypress are frameworks built on top of JavaScript (and its extension, TypeScript). You cannot write complex test scenarios, validate results, or manipulate data without mastering the basics of this language.

1.2 The Environment: Where do we write and run JS code?

The JavaScript language was initially created to run only in the browser, but today it runs everywhere. To test the examples in this course, you will use two main environments:

1. The Browser Console (DevTools) - "The Fast Playground"

The fastest way to run JS code is right in your Chrome browser.

- *How to access:* Press F12 (or right-click -> *Inspect*) and go to the **Console** tab. There you can write JavaScript code directly, press Enter, and see the result instantly. It is perfect for testing small functions, checking logic, or debugging selectors on the spot.

2. The IDE (Visual Studio Code) + Node.js - "The Professional Environment"

When writing a real automation project, you can't write hundreds of lines of code directly in the browser. You will use an **IDE (Integrated Development Environment)**, such as *Visual Studio*

Code (which you used for HTML). For your PC to understand JS outside the browser, in the following modules, we will install and use an engine called **Node.js**. In VS Code, you will write files with the .js (or .ts for TypeScript) extension and run them directly from an integrated terminal.

1.3 Variables: Memory Boxes (let vs const)

A variable is like an empty shoebox on which you put a label (the name). Inside it, you store information (the value) to reuse later. In modern JavaScript, we have two main ways to create these boxes:

1. const (The Constant - The King of Automation)

Used when the value in the "box" must **NEVER** change during the script's execution.

- *The Golden Rule:* In 90% of cases, in Playwright tests, you will use const.

```
const platformUrl = "https://certify.qualiadept.eu";
const loginButton = page.locator('#login-btn');
// If you try later to write platformUrl = "google.com", the code will
"crash" with an error, protecting you from accidental mistakes!
```

2. let (The Changing Variable)

Used ONLY when you know for sure that the value will change during execution (e.g., a counter).

```
let connectionAttempts = 0;
connectionAttempts = 1; // Allowed. The value has been updated.
```

*(Note: You might see the word **var** in very old tutorials on the internet. Avoid it! It is deprecated, "leaks" out of control structures, and causes severe memory bugs in modern applications).*

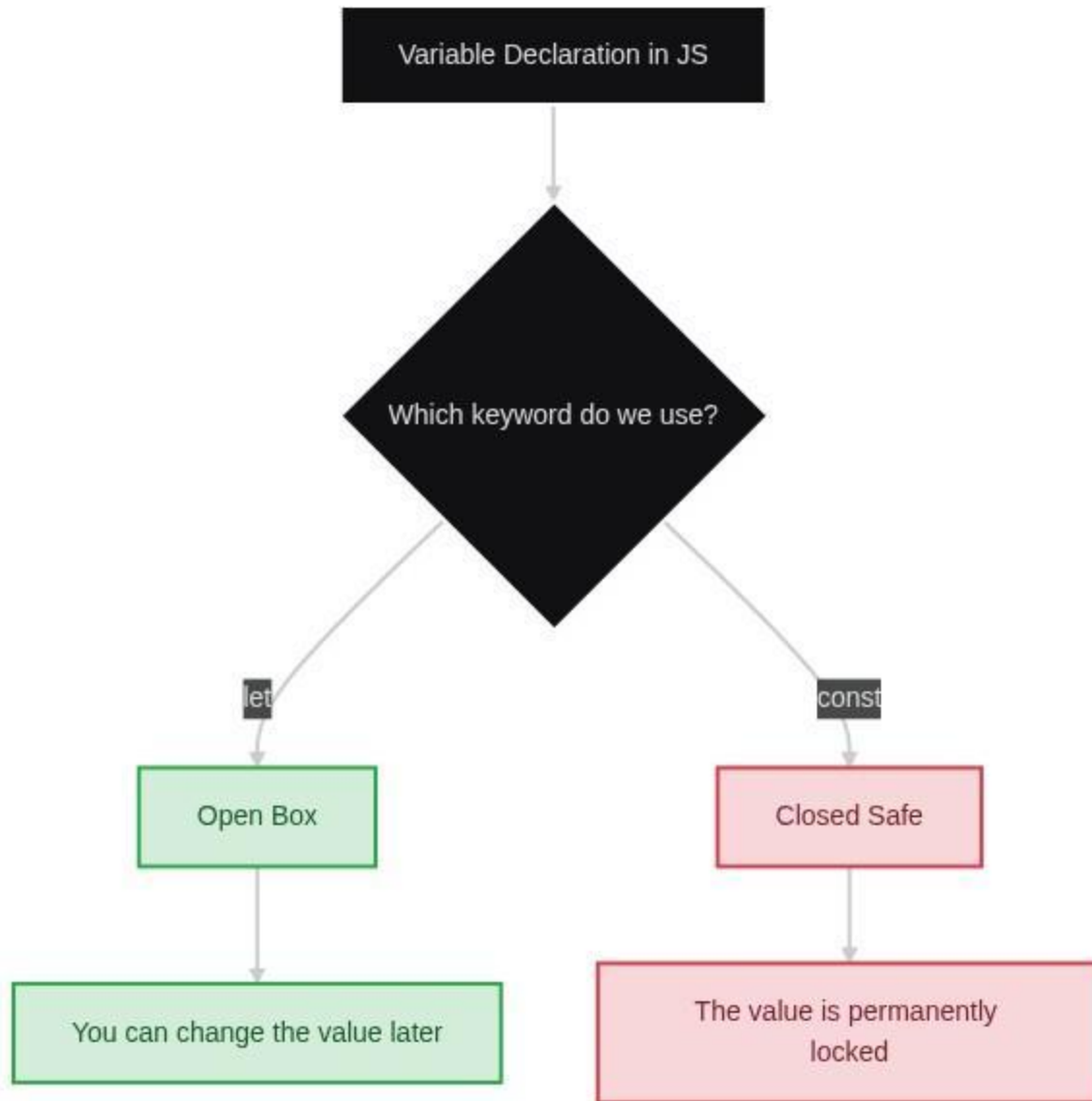


Fig. 1 - The diagram illustrates the decision flow when **declaring a variable in JavaScript**, structured across three levels:

- **Starting Point (Level 1):** A central block titled "Variable Declaration in JS".
- **Control Question (Level 2):** A diamond-shaped decision box that asks: "Which keyword do we use?". From this point, the flow splits into two branches:
 - **Left Branch (`let`):** Leads to a light green box named "**Open Box**", followed by the final explanation: "You can change the value later".
 - **Right Branch (`const`):** Leads to a light red box named "**Closed Safe**", followed by the final explanation: "The value is permanently locked".

1.4 Data Types: What do we put in the boxes?

For the robot to understand us, it needs to know exactly what kind of data it's working with. The fundamental types are:

- **String (Text):** Always written inside quotes (single '...', double "...", or backticks `...`).
 - QA Example:

```
const username = "testuser_1";
```

- **Number:** Written freely, without quotes.
 - QA Example:

```
const testTimeout = 5000; (The robot will wait 5000 milliseconds).
```

- **Boolean (True/False):** Has only two possible values: true or false.
 - QA Example:

```
const isVisible = true;
```

(Useful for validations: *I expect the button to be visible -> true*).

1.5 Control Structures: The Script's Intelligence

A smart script doesn't just run blindly from top to bottom; it makes decisions and repeats actions.

1. Decision Making (if / else)

Allows your test to react differently depending on the situation.

```
let errorMessageVisible = true;

if (errorMessageVisible === true) {
  console.log("The test failed, an error appeared on the screen!");
} else {
  console.log("The test continues, everything is fine.");
}
```

2. Repetition (The for loop)

You have a list of 10 products in the cart and want to verify that each has a price displayed. You

don't write the code 10 times! You use a for loop:

```
// Structure: for(Start; Stopping condition; Step)
for (let i = 1; i <= 3; i++) {
    console.log("Checking product number: " + i);
}
// It will print: 1, then 2, then 3.
```

1.6 Functions: Reusable Recipes

A function is a block of code with a name that executes a specific action. Instead of writing the "Login" steps 100 times in every test, you create a function that you call wherever you need it!

1. The Classic Function:

```
function addNumbers(a, b) {
    return a + b;
}
const result = addNumbers(5, 10); // result becomes 15
```

2. The Arrow Function (The Modern Standard):

This is the syntax you will see everywhere in Playwright. It is shorter and cleaner. We remove the word function and put an "arrow" => after the parameters.

```
const addNumbers = (a, b) => {
    return a + b;
};

// Automation example (simplified):
const clickButton = (selector) => {
    console.log("Playwright will click on: " + selector);
};
clickButton("#login-btn");
```

1.7 Did you know...?

💡 Did you know...

- **JavaScript is NOT Java:** The JavaScript language was created in 1995 by Brendan Eich in just 10 days! It was initially named *Mocha*, then *LiveScript*. The final name "JavaScript" was just a marketing ploy to piggyback on the massive popularity of the "Java" language at the time. As the industry saying goes: *"Java is to JavaScript as ham is to hamster"*.
- **Camel Case:** The standard way of writing variable names in JS is called *camelCase* (like a camel's humps). The first word is always lowercase, and the subsequent words start with an uppercase letter, joined together without spaces (e.g., `newUserName`, `submitButton`).
- **From animations to servers and robots:** While JS was originally invented just to make snow fall on a screen or a button blink in the browser, today, thanks to the *Node.js* platform, you can run complex servers, program physical robots, and write enterprise testing systems, all with a single language!

1.8 Applied Story: "The JavaScript Kitchen"

📄 Think of your kitchen:

- **The IDE (VS Code)** is the kitchen itself, where you have all the tools and prepare complex recipes. **DevTools Console** is the microwave: fast, but only for small tests/reheating.
- **Variables (`const` and `let`)** are your containers. The salt jar is a `const` (it always remains salt, it doesn't change). The salad bowl is a `let` (today you make a tomato salad, tomorrow you wash it and put fruit in it).
- **Data Types** are ingredients (`String` = Flour, `Number` = 3 eggs).
- **Control Structures (`if/else`)** are your decisions as a chef: *"If the soup lacks salt, I add more. Otherwise, I turn it off."*
- **The Function** is the Recipe. You have a written recipe (the function), you give it the ingredients (parameters), and it returns the finished dish (`return`).

1.9 Practical Exercises

To solve these exercises, open a new tab in Chrome, press F12, go to the **Console** tab, and write the code there, pressing Enter after each line!

Exercise 1: Correct Declaration

- Declare a constant named `appName` and give it the value "Task Tracker".
- Declare a variable (that can change) named `testsPassed` and initialize it with the value 0.
- Try to change the constant's value `appName = "Another App"`. What error does the console show you?

Exercise 2: Arrow Function for Age

Write an **arrow function** named `checkAdulthood` that receives a parameter named `age` (a Number). The function must have an if/else:

- If the age is greater than or equal to ($\geq$) 18, it should return "Adult user".
- If not, it should return "Access denied".

Call the function in the console with your age to test it!

1.10 Exercise Solutions

Solution Exercise 1:

```
const appName = "Task Tracker";
let testsPassed = 0;
// The error when modifying the constant will be: Uncaught TypeError:
Assignment to constant variable.
```

Solution Exercise 2:

```
const checkAdulthood = (age) => {
  if (age >= 18) {
    return "Adult user";
  } else {
    return "Access denied";
  }
};

console.log(checkAdulthood(20)); // Prints: Adult user
```

```
console.log(checkAdulthood(16)); // Prints: Access denied
```

1.11 🤖 Engineer Mindset in the AI Era: Your Place in the Market

🔊 QA vs. AI: The Illusion of Perfect Generated Code

With the rise of AI, many believe that *"I don't need to learn programming anymore, AI writes my code"*. This is a huge trap for a QA.

What AI does well: If you ask it *"write me a JS function that validates an email format"*, the AI will give you complex code in 2 seconds that would probably take you 20 minutes to write from scratch. It's a great assistant.

The Trap (Where AI fails): AI is prone to "logical hallucinations" and poor architectural practices if it doesn't receive perfect context. It might write an entire test scenario using only `let` instead of `const`, exposing your test data to accidental modifications (*mutation bugs*). Or it might generate an infinite for loop that will completely freeze your server or browser.

How a QA Engineer thinks: The engineer reads the AI-generated code the same way a teacher grades a student's paper. They will say: *"Thank you for the logical skeleton, but you used var instead of let, and your if/else logic doesn't cover the scenario where the form is completely empty."* The QA engineer will correct and refactor the generated function in their IDE to make it robust (*bulletproof*).

Why you will stay relevant in the market: You won't be beaten by robots that write code; you will be beaten by people who know *what to ask* the robots and how to *validate* their output. Deep knowledge of the fundamentals (Memory, Loops, Decision Logic) makes you the Architect of the testing system, while the AI remains just a fast executor. Your real value lies in validating business logic and ensuring test traceability.

Chapter 2: What do we put in the boxes? Fundamental Data Types

Learning Objectives



- Identify and use primitive data types (String, Number, Boolean).
- Understand the vital difference between undefined and null.
- Work with complex data types: Arrays (lists) and Objects (collections).
- Investigate data using the typeof operator.

2.1 Primitive Types: The Basic Ingredients

If variables (let and const) are the "boxes," data types tell us what kind of objects are stored inside those boxes. In JavaScript, as in any language, we have a few simple types, called "primitives."

- **String (Text):** Any sequence of characters. In JS, text must **always** be wrapped in quotes (single `'...'`, double `"..."`, or backticks ``...``).

```
const errorMessage = "Incorrect password!";
```

- **Number:** Unlike other languages that differentiate between integers and decimals, JS simply calls them all Number. They are never wrapped in quotes!

```
const waitTime = 5000; // 5 seconds in milliseconds
const productPrice = 99.99;
```

- **Boolean (True/False):** Has only two possible values: true or false (without quotes). It is fundamental for decision-making logic.

```
let isVisible = true;
let userLoggedIn = false;
```

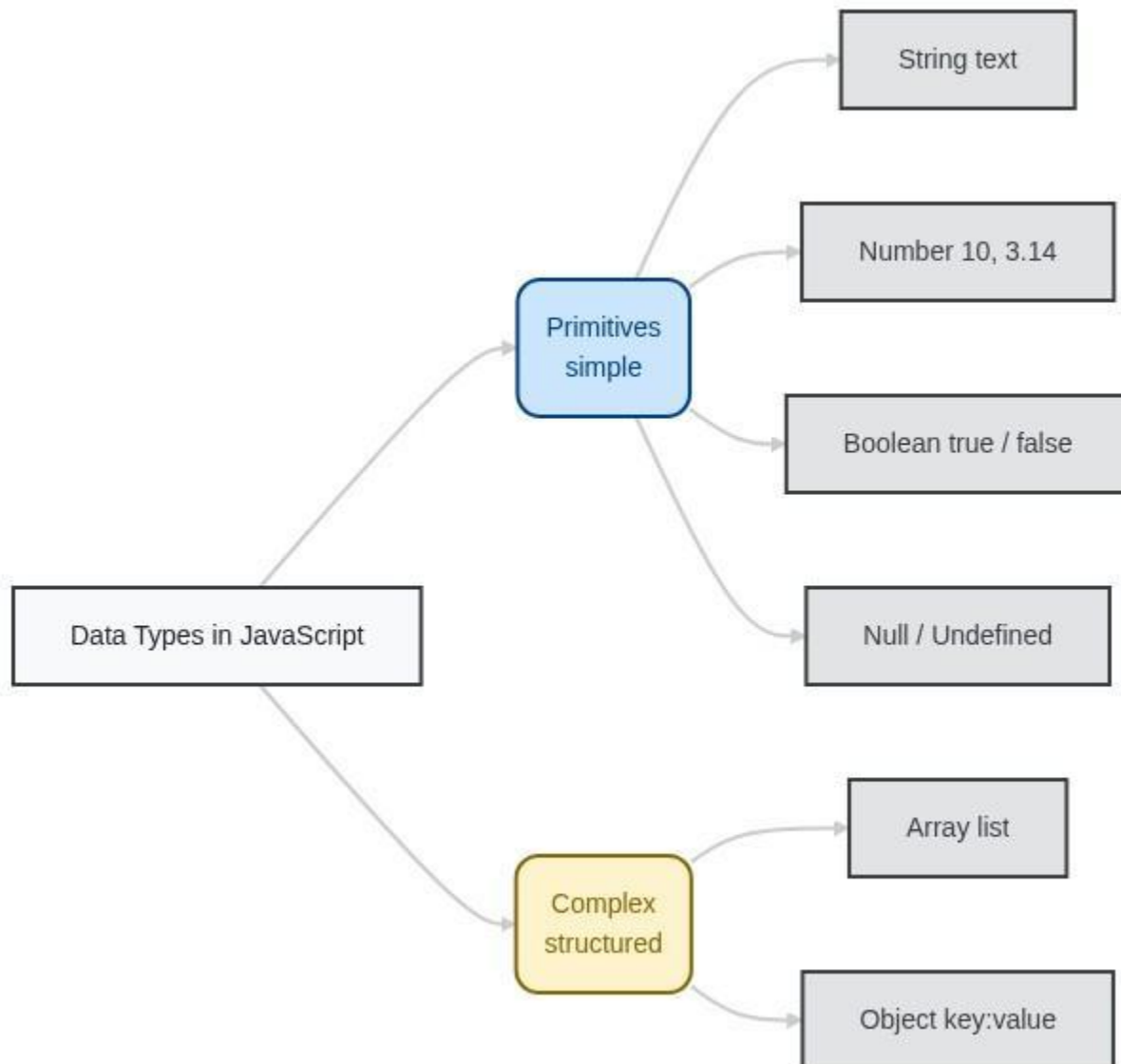


Fig. 2 - The diagram illustrates the classification of **Data Types in JavaScript**, split into two main branches:

1. **Primitive Types (simple):** Displayed in a blue block. They include:
 - **String** (text)
 - **Number** (numbers like 10 or 3.14)
 - **Boolean** (truth values: true / false)
 - **Null / Undefined** (absence of value)
2. **Complex Types (structured):** Displayed in a yellow block. They include:
 - **Array** (lists of data)
 - **Object** (key:value structures)

2.2 The Mystery of "Nothingness": undefined vs null

As a QA, you will often encounter these two concepts when your robot searches for something on the page and cannot find it.

- **undefined:** Means you have created the box, but *you haven't put anything in it yet*. The computer doesn't know what it is.

```
let targetElement;  
console.log(targetElement); // Prints: undefined
```

- **null:** Means "nothing," but it is an *intentional* nothing. You are explicitly telling the computer: "This box is empty for now."

```
let userData = null; // We will store the data here after login
```

2.3 Complex Structures: Arrays and Objects

Often, we don't want to work with a single value, but with a collection of data.

1. **The Array (List):** An ordered list of values, written inside square brackets []. Think of it as a compartmentalized box. Each element has an index (a position), but be careful: **counting always starts from 0!**

```
const supportedBrowsers = ["Chrome", "Firefox", "Webkit"];
// "Chrome" is at position 0, "Firefox" is at position 1
console.log(supportedBrowsers[0]); // Prints: Chrome
```

2. The Object: Used to describe an "entity" with multiple characteristics. It is written inside curly braces { } and contains *key: value* pairs. This is the standard format (JSON) that web applications use to communicate with each other (APIs).

```
const testUser = {
  username: "qa_expert",
  password: "Test1234!",
  isAdmin: true
};
// To access a property, we use the dot (.) syntax:
console.log(testUser.username); // Prints: qa_expert
```

2.4 Did you know...?

- **The Type Detector:** JavaScript has a magic word called `typeof` that you can use in the Console to find out what is hidden inside a variable. If you write `typeof 42`, it will answer "number". If you write `typeof "42"`, it will answer "string".
- **Everything is an Object:** Under the hood, in JavaScript, even Arrays are considered a special form of "Object" with numeric keys!
- **Beware of additions:** What happens if you add a Number to a String? `5 + "5"` in JavaScript does not equal 10, but "55". JS transforms the number into text and glues them together! This is exactly why knowing your data types is critical.

2.5 Practical Exercises

Open VS Code (create a `test.js` file and run it using `Node.js`, if you have the environment installed) or use the **Console** tab in DevTools (Chrome).

Your Mission:

- Declare a variable `age` (Number) with your actual age.
- Declare an Object named `testProduct` that has 3 characteristics: `name` (String), `price` (Number), and `inStock` (Boolean).

- Use the `console.log()` command to print the product's price (using dot syntax).
- Run the command `typeof testProduct.inStock` and see what the console tells you.

2.6 Exercise Solutions



```
let age = 28;

const testProduct = {
  name: "Wireless Headphones",
  price: 299.99,
  inStock: true
};

console.log(testProduct.price); // Will print 299.99
console.log(typeof testProduct.inStock); // Will print "boolean"
```

2.7 🤖 Engineer Mindset in the AI Era: Your Place in the Market

🔍 QA vs. AI: Data Type Confusion (Type Coercion)

- **What AI does well:** If you ask an AI assistant *"generate a user profile for a test,"* it will immediately spit out a complex, visually perfectly structured object, with dozens of properties in the correct format.
- **The Trap (Where AI fails):** In JavaScript, there is a massive trap: `==` (loose equality) vs `===` (strict equality). An AI that doesn't understand your application's exact business logic might generate a test that checks if `0 == false` (which in JS is true, because it forces them to convert types). Or it might send a fake price to the server (via API) as `"100"` (String) instead of `100` (Number). The server will crash (500 error), and the AI will spin in circles not knowing why.
- **How a QA Engineer thinks:** The engineer is obsessed with "Strict Types." They look at the AI-generated script and notice the assertion (validation) is fragile. The engineer will

correct the test to **always** use === (which checks both the value AND the data type). They know that a String is never allowed to pass as a Number in a financial system.

- **Why you will stay relevant in the market:** Your major value in the development team is not just "clicking buttons automatically," but **Data Quality Assurance**. You prevent those invisible, "silent" bugs that an AI accidentally introduces because it doesn't "understand" the fundamental difference between a string of characters representing an age and a mathematical number the app needs to calculate taxes. You bring Rigor to an often too-permissive language!

Chapter 3: Application Logic - Control Structures (if/else, for)

Learning Objectives



- Understand how code executes (top-down control flow) and how you can divert it.
- Write decision-making statements (if / else if / else) to handle different testing scenarios.
- Use the for loop to automatically repeat blocks of code.
- Combine Arrays (from the previous chapter) with for loops to iterate through data lists.

3.1 Making Decisions: The if / else Statement

Normally, JavaScript reads your code just like you read a book: line by line, from top to bottom. Control structures allow us to change this order.

The simplest and most used decision structure is the **if / else statement**. It tells the computer: *"If this condition is true, execute block A. Otherwise, execute block B."*

```
let elementVisible = true;

if (elementVisible === true) {
  // This code executes ONLY if the condition in the parentheses is true
  console.log("Click the button!");
} else {
  // This code executes if the condition is false
  console.log("Wait for the element to appear...");
}
```

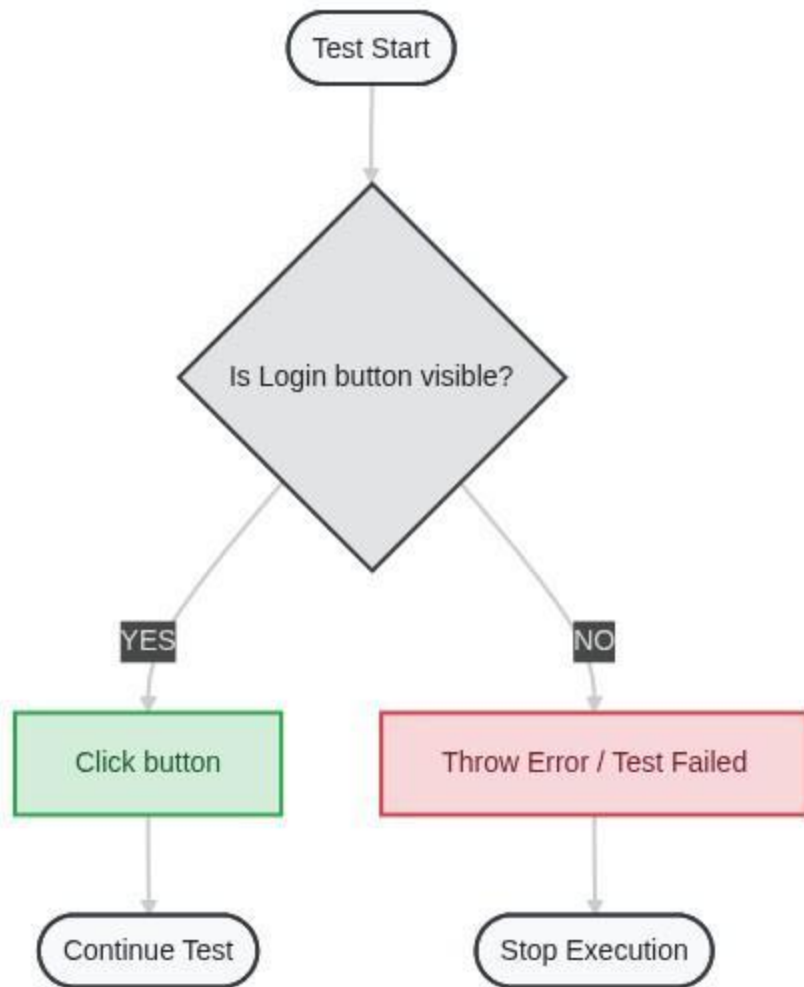


Fig. 3 - The diagram visualizes conditional execution (**if/else logic**) within an automated test script:

- **Starting Point:** The test initiates and checks a condition inside a decision diamond: "*Is Login button visible?*".
- **The if branch (True / YES):** If the button is visible, the script executes the action Click button (styled in green), and then moves forward to Continue Test.
- **The else branch (False / NO):** If the button is missing, the script branches to the error handling block Throw Error / Test Failed (styled in red), leading immediately to Stop Execution.

3.2 Multiple Branches: else if

In real QA life, things aren't just black and white. What happens when we test an API and want to check the response (Status Code)? We need more options, which is where else if comes in:

```
const statusCode = 404;

if (statusCode === 200) {
    console.log("Test Passed: Page loaded correctly.");
} else if (statusCode === 404) {
    console.log("Test Failed: Page not found (Client Error).");
} else if (statusCode === 500) {
    console.log("Test Failed: The server crashed completely!");
} else {
    console.log("Unknown error.");
}

// Because the statusCode is 404, the console will print the second
// message.
```

3.3 Repetition: The for Loop

One of the core principles in programming is **DRY (Don't Repeat Yourself)**. If you want to click on 5 identical buttons, you don't write the click command 5 times. You use a "loop" to automate the repetition.

The for loop structure has 3 parts inside the parentheses:

- **Start (Initialization):** Where do we start counting from? (e.g., let i = 1)
- **Stopping Condition:** Until when do we repeat? (e.g., i <= 5)
- **Step (Increment):** By how much do we increase the counter at the end of each round? (e.g., i++ means we increase it by 1).

```
for (let i = 1; i <= 3; i++) {
    console.log("Executing action number: " + i);
}

// Console Output:
// Executing action number: 1
// Executing action number: 2
// Executing action number: 3
```

3.4 Iterating through Lists (Combining Loops with Arrays)

This is where everything becomes extremely relevant for an Automation Engineer. Imagine you have an Array with usernames and you want to test the login for each one. You will use a for loop to go through each element, using the .length property (which tells us how many elements the list has).

```
const testUsers = ["admin", "editor", "reader"];

// i starts from 0 because Array indexes start at 0!
for (let i = 0; i < testUsers.length; i++) {
  console.log("Testing login for user: " + testUsers[i]);
}
```

3.5 Did you know...?



- **The Infinite Loop (Programmer's Nightmare):** If you mess up the "Stopping Condition" in a for loop, the loop will never end. The computer will repeat the action infinitely until it runs out of memory and crashes (freezes). For example, if you forget to write `i++`, the variable `i` remains 1 forever, and the condition `1 <= 5` will always be true.
- **Truthiness & Falsiness:** In JS, an if condition doesn't strictly need a boolean true or false. Certain values are directly evaluated as false (falsy values): 0, "" (an empty string), null, and undefined. Thus, `if ("") { ... }` will never execute.

3.6 Practical Exercises

Open DevTools (Console tab) in your browser or use a file in VS Code to solve the following exercises:

Mission 1: Password Check

- Declare a variable `incorrectPassword` and assign it the value `true`.
- Write an `if/else` block. If `incorrectPassword` is `true`, print "Show UI error message". Otherwise, print "Redirect to Dashboard".

Mission 2: Table Search

- Write a `for` loop that counts from 1 to 4.
- Inside the loop, use `console.log()` to print: "Checking row number X" (where X is your counter variable).

3.7 Exercise Solutions

Solution Mission 1:

```
let incorrectPassword = true;

if (incorrectPassword === true) {
    console.log("Show UI error message");
} else {
    console.log("Redirect to Dashboard");
}
```

Solution Mission 2:

```
for (let i = 1; i <= 4; i++) {
    console.log("Checking row number " + i);
}
```

3.8 Engineer Mindset in the AI Era: Your Place in the Market

QA vs. AI: Infinite loops and blind logic at Edge Cases

- **What AI does well:** Code generators are fantastic at quickly writing boring syntax for repetitive structures. If you tell it *"iterate through an array of products and click"*, it will immediately give you a syntactically correct for loop.
- **The Trap (Where AI fails):** AI often suffers from "context myopia." It writes the ideal code (Happy Path). But what happens if the array of products comes back empty from the server? The AI might generate if / else logic that assumes the product always exists. When you run the test in the Pipeline (CI/CD) and the network is slow, the array is empty, the test fails, and the AI suggests you put in artificial pauses (sleeps/timeouts).
- **How an engineer thinks:** A QA engineer applies *Boundary Value Analysis* directly to the code. They look at the AI-generated for loop and add "defensive" validation before it: `if (products.length === 0) { console.log("Test failed: Products did not load"); return; }`. The engineer doesn't let the loop start blindly on invalid data.
- **Why you will stay relevant in the market:** Your value doesn't lie in knowing the syntax of a for loop by heart, but in **Critical Thinking**. The AI encodes the algorithm; you are the one asking: *"What happens if this condition is negative? What happens if I receive null instead of an array?"*. You create the robust testing architecture that a machine simply cannot plan in advance.

Chapter 4: Code Reusability - Classic Functions and Arrow Functions

Learning Objectives



- Understand what a function is and how it helps you respect the DRY (Don't Repeat Yourself) principle.
- Declare and call classic functions using parameters and the return keyword.
- Write modern functions using the Arrow Function syntax (`=>`), the absolute core standard in Playwright.
- Transition from "monolithic" code (written all together) to modular and clean code.

4.1 What is a function and why do we need it?

Imagine you are writing an automated test for an online store. You need to log in at the beginning of the test for adding to the cart, at the beginning of the test for checkout, and at the beginning of the test for checking the account history.

Are you going to write the same 5 steps of code (enter email, password, click button) dozens of times? No!

A **function** is a reusable block of code that groups multiple instructions under a single name. You write it once and "call" it whenever you need it, from anywhere in your application.

QA Analogy: A function is a cooking recipe.

- **Parameters (Ingredients):** What you give the function at the beginning so it has something to work with (e.g., user and password).
- **Function Body (Cooking):** The actual code that executes the logical steps inside.
- **Return (The final dish):** The processed result the function gives back to you after it finishes

(e.g., a signal that you successfully logged in).

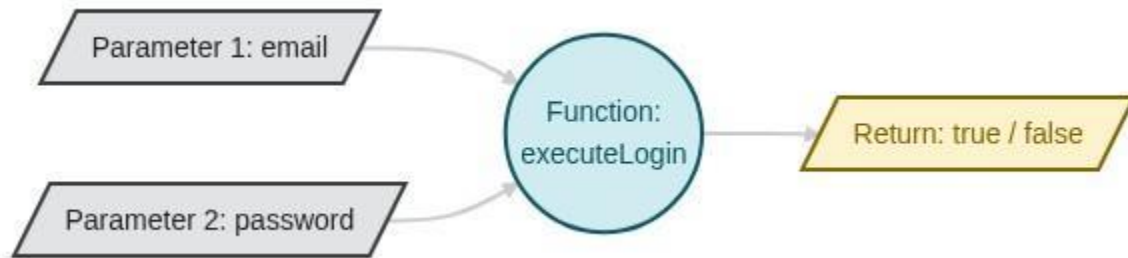


Fig. 4 - The diagram visualizes the core **Anatomy of a Function** in JavaScript using the input-output model:

- **Inputs (Parameters):** On the left side, two parallel slanted boxes represent the input data passed to the function: "*Parameter 1: email*" and "*Parameter 2: password*".
- **Processing (The Function):** Arrows point from the inputs into a central blue circular block titled "*Function: executeLogin*". This acts as a "black box" where the validation logic runs.
- **Output (The Result):** An arrow leaves the function block and points to a yellow slanted box on the right, representing the final output: "*Return: true / false*".

4.2 The Classic Function

The traditional syntax in JavaScript (the one the language started with) uses the function keyword.

```

// Function declaration
function calculateTotalPrice(basePrice, tax) {
  let finalPrice = basePrice + tax;
  return finalPrice; // Returns the calculated value back to whoever
called the function
}

// Calling (using) the function
let cartTotal = calculateTotalPrice(100, 19);
console.log("The total to pay is: " + cartTotal); // Prints in console: 119
  
```

Notice that `basePrice` and `tax` are temporary variables (parameters) that only "catch" the values 100 and 19 at the exact moment the function is executed on the calling line.

4.3 The Arrow Function - The Modern Standard

Although classic functions are still perfectly valid and widely used, in 2015 JavaScript introduced a much shorter and cleaner syntax: the **Arrow Function** (`=>`).

For you, as a future QA Automation Engineer, this is **the most important syntax in this module**. Playwright, Cypress, and all modern testing frameworks are written relying almost exclusively on Arrow Functions.

How do we transform the classic function above into an arrow function? We remove the word `function`, add an arrow `=>` between the parameters and the curly braces, and store it nicely in a constant:

```
const calculateTotalPrice = (basePrice, tax) => {  
  let finalPrice = basePrice + tax;  
  return finalPrice;  
};  
  
console.log(calculateTotalPrice(50, 5)); // Prints: 55
```

Why do we love them in Playwright?

When you reach Module 4, absolutely every test you write will look exactly like this:

```
test("Verify the Login button", async ({ page }) => {  
  // Here comes your test's function body! All your E2E logic is written  
  here.  
});
```

See the arrow? The test itself is simply an *Arrow Function* that the Playwright framework executes for you!

4.4 Did you know...?



- **Implicit Return:** If your arrow function is extremely simple and has a single line of code that just returns a result, you can completely drop the curly braces {} and the return keyword.
Ex: `const double = (number) => number * 2;`. Super short and elegant!
- **Functions without parameters:** If a function doesn't need any input "ingredient" to do its job, the parentheses remain empty:
Ex: `const greet = () => { console.log("Hello QA!"); };`
- **They are "First-Class Citizens":** In JavaScript, you can do with functions exactly what you do with any other variable or value. You can save them, put them in Arrays, or, much more interestingly, you can pass a function as a parameter *inside another function* (an advanced concept called a *Callback*, on which asynchronous JavaScript heavily relies).

4.5 Practical Exercises

Open DevTools (Console tab) in the browser or the integrated terminal in VS Code to practice:

Mission 1:

Write a classic function named `greetUser` that receives a parameter name (a String) and uses `console.log` to print: "Welcome, [name]!". Call the function passing your name as a parameter.

Mission 2 (Validation Logic):

Write an **arrow function** named `checkAccess` that receives two parameters: `isAdmin` (Boolean) and `isLoggedIn` (Boolean).

- In the function body, use an `if` (structure learned in the previous chapter). If both conditions are true (use the logical operator `&&`), the function must return the String "Access granted to Dashboard".
- Otherwise (else), return "Access denied".
- Call the function in the console with the values `true, true` and then with `false, true`, using `console.log()` to see the result live.

4.6 Exercise Solutions

☑ Solution Mission 1:

```
function greetUser(name) {  
    console.log("Welcome, " + name + "!");  
}  
  
greetUser("Alex");
```

☑ Solution Mission 2:

```
const checkAccess = (isAdmin, isLoggedIn) => {  
    if (isAdmin === true && isLoggedIn === true) {  
        return "Access granted to Dashboard";  
    } else {  
        return "Access denied";  
    }  
};  
  
console.log(checkAccess(true, true)); // Will print: Access granted to  
Dashboard  
console.log(checkAccess(false, true)); // Will print: Access denied
```

4.7 🗣️ Engineer Mindset in the AI Era: Your Place in the Market

🗣️ QA vs. AI: Modular Architecture vs. Spaghetti Code

- **What AI does well:** Assistants like GitHub Copilot or Cursor are extremely fast at generating functional code to fulfill a specific task. If you tell it *"write me a test for purchasing a product"*, it will write absolutely all the steps from top to bottom.
- **The Trap (Where AI fails):** By their nature, LLMs tend to generate "monolithic" and repetitive code if they don't receive strict architectural context and instructions. If you have 50 Checkout tests automatically generated by AI, you might find that the long logic for filling

out the shipping form (Name, Address, Card) was duplicated identically across all 50 test files. If tomorrow the developers change the HTML selector for the "County/State" field, you will have to manually search and modify the code in 50 different places. This is considered *unmaintainable* code.

- **How a QA engineer thinks:** The engineer acts like an Architect. They read the AI-generated scenario and immediately notice the dangerous repetition. They will take the address-filling logic and extract it into a separate **Arrow Function** (e.g., `const fillAddress = (city, county) => { ... }`) which they will export and call as a simple command in the 50 tests. Now, if the application interface changes, the engineer updates the selector *in a single place* (in the body of their function), and all 50 tests are "fixed" instantly!
- **Why you will stay relevant in the market:** Your huge commercial value to a company derives from your ability to build **scalable Frameworks**. Thinking in functions, modules, and classes (like the future *Page Object Model* we will learn in Module 6) is a strategic vision that AI cannot plan alone at a macro level for a continuously developing enterprise-type application. You are not just a "writer of automated commands"; you ensure the long-term sustainability of the code, drastically reducing the maintenance time of the entire team.

Chapter 5: Session 3 Assignment & Extra Practice (Advanced Level)

5.1 Homework Assignment (Session 3 Project)

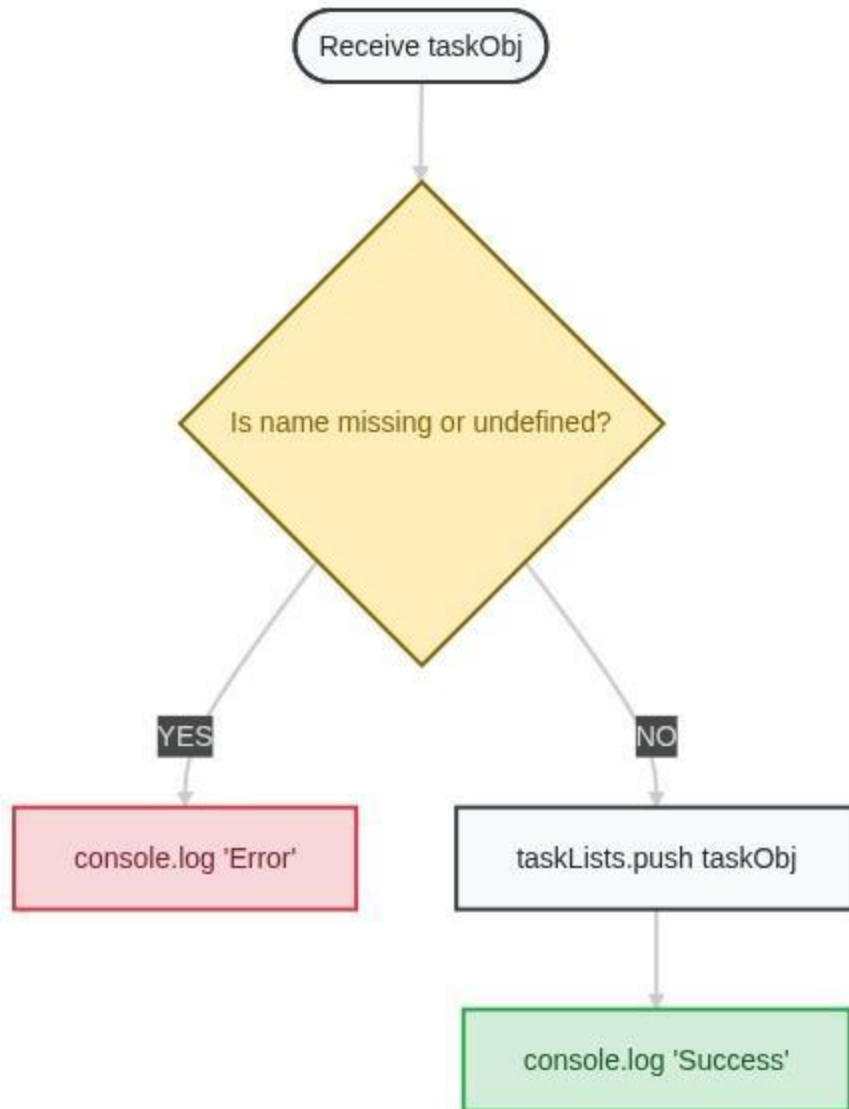


Fig. 5 - The diagram visualizes the **Acceptance Criteria** for the validation function required in the homework:

- **Trigger:** The function triggers upon receiving an object as an argument: taskObj.
- **Validation Condition:** The object structure is checked inside a yellow decision box: "Is name missing or undefined?".
- **Error Case (YES):** If the property is missing, the code branches to the left and executes console.log 'Error' (styled in red), terminating the flow.
- **Success Case (NO):** If the object is valid, it branches to the right:
 - It appends the object to the collection via taskLists.push taskObj.
 - It then confirms the operation by running console.log 'Success' (styled in green).

green).

Your Task (The Business Task):

So far, you have built the "skeleton" (Session 1) and the "design" (Session 2) of our **"Task Tracker"** application. Now it's time to bring it to life by building the first part of its "brain" (core logic). You need to write a JavaScript script that stores and processes the task list, validating the data *before* it is displayed on the screen.

 **Execution Note:** You will write this code either in a separate **app.js** file (and link it in your HTML with `<script src="app.js"></script>` at the **end** of the `<body>`), or directly in the Console tab of Chrome DevTools for quick testing.

Technical Requirements (Acceptance Criteria for validation):

-  **E2E Data Structure (Arrays & Objects):**
 - Declare a constant named `taskList` that is an Array.
 - Initialize this Array with the two tasks already existing in your HTML table from previous Sessions. Each task must be an Object with 3 properties: `id` (Number), `name` (String), and `completed` (Boolean).
 - *Example:* `[{ id: 1001, name: "Set up HTML Boilerplate", completed: true }, ... ]`
-  **Validation Arrow Function (Business Logic):**
 - Write an Arrow Function named `processNewTask`. This receives a single parameter: `taskObj` (an Object).
 - **Strict Validation (if/else):** Check if the task name is missing ("") OR is undefined.
 - If the data is invalid, use `console.log` to display: "Error: The task must have a name!".
 - If the data is correct, add the new object to the `taskList` Array (using the `push()` method) and display: "Task added successfully!".
-  **Reporting Loop (Repetition):**
 - After testing the function above (manually adding a valid task, e.g.: `id: 1003`), write a for loop.
 - This loop must iterate through the entire `taskList` Array and use `console.log` to list in the console only the *name* of each existing task in memory at that moment.

5.2 Bonus Challenge (Optional): The API "Falsy" Trap

The Scenario:

Your Task Tracker app is in production. You receive an API response from the server with a status code. You need to write a simple logging function: if the status is 200, you print "OK", otherwise you print "Server Error".

A colleague writes this code:

```
const checkServer = (statusCode) => {
  if (!statusCode) { // Relies on the Falsy concept
    console.log("Server Error or No Response");
  } else {
    console.log("Status OK: " + statusCode);
  }
};
```

Your automated test runs and gives a *False Positive* (it passes even though it's broken). One day, the server returns a valid code that your colleague didn't foresee (e.g., the code 0 for the "waiting" state of an internal service). Because 0 is directly evaluated as *Falsy* in JS, the colleague's function throws an error, even though the server is working!

Your Mission:

How do you rewrite that if/else block applying a **strict type check (=== and typeof)**? Your function, named `checkServerStrict(statusCode)`, must return "Status OK" ONLY if the received parameter is of type Number. If it receives null, undefined, or any String (e.g., "200"), it must return "Server data format error".

5.3 Solution Keys (Answers & Explanations)

   Don't cheat! Read this only after you've tried to write the code yourself.

Homework Solution:

```
// 1. Array of Objects (Our temporary database)
const taskList = [
  { id: 1001, name: "Set up HTML Boilerplate", completed: true },
  { id: 1002, name: "Understand Client-Server Architecture", completed:
false }
];

// 2. Validation Function (Arrow Function)
const processNewTask = (taskObj) => {
  // We use || (logical OR) and === (strict equality)
  if (taskObj.name === "" || taskObj.name === undefined) {
    console.log("Error: The task must have a name!");
  } else {
    taskList.push(taskObj);
    console.log("Task added successfully!");
  }
};

// We simulate the user's action by adding a wrong task and a correct one:
processNewTask({ id: 1003, name: "", completed: false }); // Prints: Error
processNewTask({ id: 1004, name: "Write first Playwright test", completed:
false }); // Prints: Success

// 3. Reporting loop (The list now has 3 valid elements)
console.log("--- Task Report ---");
for (let i = 0; i < taskList.length; i++) {
  console.log("Task " + taskList[i].id + ": " + taskList[i].name);
}
```

☒ Bonus Challenge Solution (Strict Checking):

```
const checkServerStrict = (statusCode) => {
  // Instead of relying on Falsy, we clearly validate the data contract:
  It must be a Number!
  if (typeof statusCode === "number") {
    return "Status OK: " + statusCode;
  } else {
    return "Server data format error";
  }
};
```

```
console.log(checkServerStrict(0));           // Returns "Status OK: 0" (We
fixed the colleague's bug!)
console.log(checkServerStrict("200"));       // Returns "Server data format
error" (Strings are not allowed)
```

🐛 **Congratulations!** By integrating this script into app.js and linking it to your HTML from Sessions 1 and 2, you have just laid the foundation for your first functional application (Frontend-Logic). When we move to Playwright, your robots will interact with the DOM to trigger exactly these strict checking functions that you wrote today!